The leanquote package

Quote Lean declarations in LaTeX by name

Stefan Langerman Erik D. Demaine

Version 0.10, 2026-10-25
https://github.com/slef/leanquote

Abstract

leanquote typesets declarations of a Lean 4 file in a LaTeX document by name: `\leanfile{Abstract.lean}` followed by `\leanquote{main_theorem}` prints the declaration `main_theorem` exactly as it appears in the Lean source, read at compile time. The prose stays in LaTeX, and the code is never retyped or pasted. A misspelled name is an error, and the package warns when the Lean file has changed since Lean last compiled it. It needs nothing on the Lean side beyond the usual `lake build`; an optional exporter lets Lean itself report the exact extent of each declaration. It works with pdfLaTeX, LuaLaTeX, and XeLaTeX.

1 Why

Papers that use a formalization often reproduce its definitions and main theorem statements, so that a reader can check that the formal statements say what the paper claims. Pasting code by hand invites the copy in the paper to drift from the code that Lean checked. With leanquote the paper names the declarations it wants, and LaTeX copies them from the Lean file each time the paper is compiled. The package was written for papers that present a short, human-readable `Abstract.lean`, containing the definitions and main statements of a formalization and proved through a bridge to the full development, but it works with any Lean 4 file.

2 Usage

```
\usepackage{leanquote}
\leanquotesetup{project=../lean/}            % root of the Lake project (optional)

\leanfile{TranslationTiling/Abstract.lean}  % current file
A prototile is a finite list of points of $\mathbb{Z}^3$:
\leanquote{Point, Prototile}
```

produces (with the file shipped in the package's `example` directory):

A prototile is a finite list of points of $\mathbb{Z}^3$:

```
abbrev Point := Fin 3 → ℤ
```

```
abbrev Prototile := List Point
```

\leanfile[⟨*options*⟩]{⟨*file*⟩} sets the current file for the following **\leanquote** commands. Options given here apply to all of them. The path is relative to **project**, if set. The same command is also available as **\leanquotefile**. If the document already defines **\leanfile** (some blueprint macro files do), leanquote leaves that definition alone, warns, and **\leanquotefile** is used instead.

\leanquote[⟨*options*⟩]{⟨*names*⟩} typesets the comma-separated declarations ⟨*names*⟩ of the current file, together, in one listing.

\leanfilequote[⟨*options*⟩]{⟨*file*⟩}{⟨*names*⟩} does the same for an explicitly given file.

\leanquotesetup{⟨*options*⟩} sets options for the rest of the document (or the current group).

Names may be short (**Tiles**) or qualified (**TranslationTiling.Abstract.Tiles**). A qualified name matches a declaration inside the corresponding **namespace**: either name may be a dotted suffix of the other. An unknown name is an error; an ambiguous one is a warning, and the first match is used.

With the **doc** option, the doc comment is included:

```
\leanquote[doc]{Tiles}
```

```
/-- `P` tiles `ℤ³`: for some set `T` of translations, every point `x` lies in
      exactly
one translate `t + P` with `t ∈ T`, i.e., for exactly one `t ∈ T` we have `x - t ∈
      P`. -/
def Tiles (P : Prototile) : Prop :=
  ∃ T : Set Point, ∀ x : Point, ∃! t : Point, t ∈ T ∧ x - t ∈ P
```

3 Options

Option	Default	Meaning
doc	false	also show each declaration's doc comment
box	true	put the listing in a light frame (**leanquotebox**)
logo=⟨*file*⟩	none	image in the frame's top-right corner
style=⟨*name*⟩	**leanquote**	listings style used for the code
project=⟨*dir*⟩**/**	empty	root of the Lake project
olean=⟨*file*⟩		compiled file to compare with, if not at the standard place
verify=⟨*mode*⟩	**auto**	**auto**, **true** or **false**; see Section 5
root=, **source=**		for explicit exports: where the recorded source is
any other key		passed to **\lstinputlisting**

The logo. With `logo=`⟨*file*⟩, the image appears in the top-right corner of each box, followed by `\leanquotelogomark`, a small "TM" in the blue of the Lean logo. The Lean trademark policy (`https://lean-lang.org/trademark-policy`) asks that the Lean logo always be accompanied by a trademark symbol; for other logos, redefine `\leanquotelogomark` as empty. The Lean logo is not included in the package.

4 How declarations are found

The package reads the Lean file with LaTeX's own file input and uses the standard layout of Lean code. A declaration starts at a line in column 0 of the form

`[@[`⟨*attributes*⟩`]] [`⟨*modifiers*⟩`]` ⟨*keyword*⟩ ⟨*name*⟩ `...`

where ⟨*keyword*⟩ is one of `def`, `theorem`, `lemma`, `abbrev`, `structure`, `inductive`, `instance`, `class`, `axiom`, `opaque`, and the modifiers are `private`, `protected`, `noncomputable`, `partial`, `unsafe`, `nonrec`. Attribute lines just above it are included, and so is the doc comment if `doc` is set. The declaration continues while lines are blank, indented, or start with `|`, `termination_by`, `decreasing_by` or `deriving`; trailing blank lines are dropped. This matches the layout used by Mathlib and by most Lean code, including `match` alternatives and `inductive` constructors.

5 What has Lean checked?

LaTeX does not run Lean, but it can read what Lake leaves behind. leanquote marks a quote, in the corner of its box, when it is not known to be fully checked, since a warning in the log is easily missed. Without a box, the mark appears in red above the code.

Mark	Meaning
UNVERIFIED	the `.lean` file is newer than its compiled `.olean` (edited since the last build), or its last `lake build` failed (Lake deletes the `.olean` but keeps its `.trace`); with `verify=true`, also when there is no compiled file
USES SORRY	the declaration depends on `sorry`, anywhere in its dependencies (needs an export, Section 7)
SORRY IN FILE	Lake's build log for the file reports a `sorry` (without an export)
EXTRA AXIOMS	the declaration depends on axioms beyond `propext`, `Classical.choice` and `Quot.sound`, for instance through `native_decide` (needs an export)

The compiled files are looked for under `project` (`.lake/build/lib/lean/...`). In short: by default, the code shown is what Lean last compiled successfully; with an export, it is also known not to depend on `sorry` or on non-standard axioms.

The `verify` option chooses how strict to be:

	compiled, up to date	stale or failed build	no compiled file
`verify=auto` (default)	checks as above	UNVERIFIED	note in the log
`verify=true` (or `verify`)	checks as above	UNVERIFIED	UNVERIFIED
`verify=false`	nothing	nothing	nothing

The default catches the common mistakes, editing the Lean file or breaking its build and then recompiling the paper, and stays silent where no compiled file is available (Section 6). With `verify=true`, those quotes are marked too:

UNVERIFIED

```
def Adjacent (a b : Point) : Prop :=
  ∑ i, |a i - b i| = 1
```

(The example file shipped with the package has no compiled `.olean`.) The log explains each mark, for example:

```
Package leanquote Warning: ../lean/TranslationTiling/Abstract.lean is newer
(leanquote)  than its compiled file ... The code shown may not be what Lean
(leanquote)  checked (the quote is marked UNVERIFIED): run lake build.
```

What is not detected. Without an export, a `sorry` in another file that the quoted declaration depends on (say, in the bridge to the full development) is not seen, because Lake reports a `sorry` only in the file that contains it; the export's axiom list covers all dependencies. And if an imported file is broken while the quoted file is unchanged, the quoted file keeps its last successful `.olean`, so its quotes still show code that Lean checked, against the previous version of its imports.

The old option name `check` is also accepted. The marks' appearance can be changed by redefining `\leanquotebadge{label}` (in the box) and `\leanquotebadgetext{label}` (without a box).

6 Publishing: arXiv, CI, Overleaf

arXiv compiles the LaTeX source itself, and so do continuous-integration builds and online editors such as Overleaf, usually without a Lean build. There, the `.lean` files are read as usual (include them, and `leanquote.sty`, in the upload), but there is no `.olean` to compare with, so with the default `verify=auto` nothing is marked: those builds simply cannot check. The check happens where Lean is built:

1. run `lake build`, then compile the paper locally, and make sure no quote is marked UNVERIFIED and the log has no leanquote warning;
2. then upload, or push to the CI or online editor.

If a CI pipeline does build Lean before the paper, set `verify=true` there, so that a missing compiled file is noticed as well.

7 Optional: exact boundaries and axioms from Lean

For files laid out unusually, Lean itself can report where each declaration starts and ends. From the root of the Lake project, run the exporter shipped with the package:

```
path/to/leanquote/leanquote-export TranslationTiling.Abstract [More.Modules ...]
```

It builds each module and writes `.lake/build/leanquote/TranslationTiling/Abstract.leanquote`, inside the build directory. From then on, `\leanquote` finds this export automatically and uses Lean's declaration ranges; nothing changes in the document. The export also lists, for each declaration, the axioms it depends on (as `#print axioms` reports them), which gives the USES SORRY and EXTRA AXIOMS marks. The export also records the source as compiled. If the `.lean` file is edited afterwards, the package warns and falls back to reading the `.lean` file directly, so the document always shows the current code. The exporter is a single Lean file (`lean/LeanQuoteExport.lean`) run with `lake env lean -run`; nothing is added to the Lean project. An export placed next to the `.lean` file is also found, and an export can be quoted explicitly with `\leanfilequote{Abstract.leanquote}{...}`.

8 Engines and symbols

The `leanquote` listings style typesets about a hundred of Lean's Unicode symbols ($\mathbb{N}$ $\mathbb{Z}$ $\mathbb{R}$ $\to$ $\forall$ $\exists$ $\sum$ $\bigcup$ $\leq$ ..., Greek letters, sub- and superscripts) as math, under pdfLaTeX through listings' `literate` mechanism, and under LuaLaTeX and XeLaTeX by making these characters active inside listings. Other symbols can be added with `literate` in a style of your own (pdfLaTeX), or by choosing a monospace font that contains them (LuaLaTeX, XeLaTeX). The package loads listings, xcolor, graphicx, amssymb, and tcolorbox with its `skins` and `breakable` libraries. It writes one temporary file ⟨*jobname*⟩`-leanquote-`⟨*n*⟩`.lean` per listing.

9 Texlish

Texlish passes LaTeX commands through, and its one-argument syntax reads naturally:

```
::use leanquote
\leanfile: Abstract.lean
\leanquote: main_theorem
```

With an optional argument, use braces: `\leanquote[doc]{main_theorem}`.

10 Limitations

- Without an export, declaration boundaries follow the layout rules above; code that puts continuation lines in column 0 (other than the listed keywords) is cut short.
- Namespaces are not tracked: two declarations with the same short name in different namespaces give an ambiguity warning; quote them with a name that distinguishes them.
- The listings language is called `leanquote-lean`, so that a document's own `\lstdefinelanguage{Lean4}` does not interfere with it.
- The compiled check relies on file modification dates; copying a project without preserving them can cause false warnings or hide stale code.

11 Related tools

Several tools include code from source files in documents; as far as we know, none quotes Lean declarations by name in an ordinary LaTeX document or checks the quoted code against the compiled module.

- **Marker-based inclusion in LaTeX.** catchfilebetweentags (`\ExecuteMetaData`), listings line ranges with marker comments, piton ranges, and tagsnip include the code between markers that the author adds to the source; minted's `rangeregex` selects code by a regular expression, and Pygments has a Lean 4 lexer. These work for any language but need markers or patterns, and do not know Lean's declarations.
- **Lean-native documents.** Verso and SubVerso, by the Lean FRO, include anchored regions of Lean modules in documents written in Verso, with Lean's own highlighting, and fail when a code block no longer matches its module; Verso's manual genre also produces TeX. They are the natural choice for documents written in Lean; leanquote is for papers written in LaTeX.
- **Blueprints.** leanblueprint's `\lean{...}` links informal statements to Lean declarations by name, and `checkdecls` checks that the names exist; LeanArchitect generates blueprint text from annotated Lean declarations (`\inputleannode{...}`). Both connect LaTeX and Lean by name, without quoting code, and can be used together with leanquote.
- **Other proof assistants.** Isabelle's document preparation and the cartouches tool (formerly isasnips), which names snippets after declarations; Agda's LaTeX backend combined with catchfilebetweentags; and Alectryon for Rocq. There the checker itself generates the document, which keeps code and proofs in sync.
- **Lean listings style.** Lean's documentation provides `lstlean.tex`, a listings definition of Lean with a Unicode table, which inspired ours.

The idea of including Lean snippets in papers to keep them in sync with the code was raised on the Lean Zulip by Eric Wieser in 2022 (thread "MathemaTeX").

12 Acknowledgments

The package and this documentation were produced with the help of Claude Opus 5.5 (Anthropic), working under the authors' direction.

13 License and history

leanquote is released under the MIT License. It grew out of a position paper by the authors on presenting the formal statements of machine-checked results to human readers.

0.10 (2026-10-25) Failed builds marked UNVERIFIED; USES SORRY, SORRY IN FILE and EXTRA AXIOMS marks; axioms in exports.

0.9 `verify=auto` (new default), `true`, `false`; publishing section.

0.8 Trademark mark after the logo (`\leanquotelogomark`).

0.7 UNVERIFIED mark and `verify` option; `\leanquotefile` and a safe `\leanfile`; listings language renamed `leanquote-lean`.

0.6 Timestamps compared with LaTeX3's `\file_compare_timestamp`; related tools.

0.5 LuaLaTeX and XeLaTeX support; documentation.

0.4 Renamed from leandecl; `\leanfile` and `\leanquote`.

0.3 Optional exports used automatically when up to date.

0.2 Direct reading of `.lean` files; compiled check.

0.1 First version, based on exports.